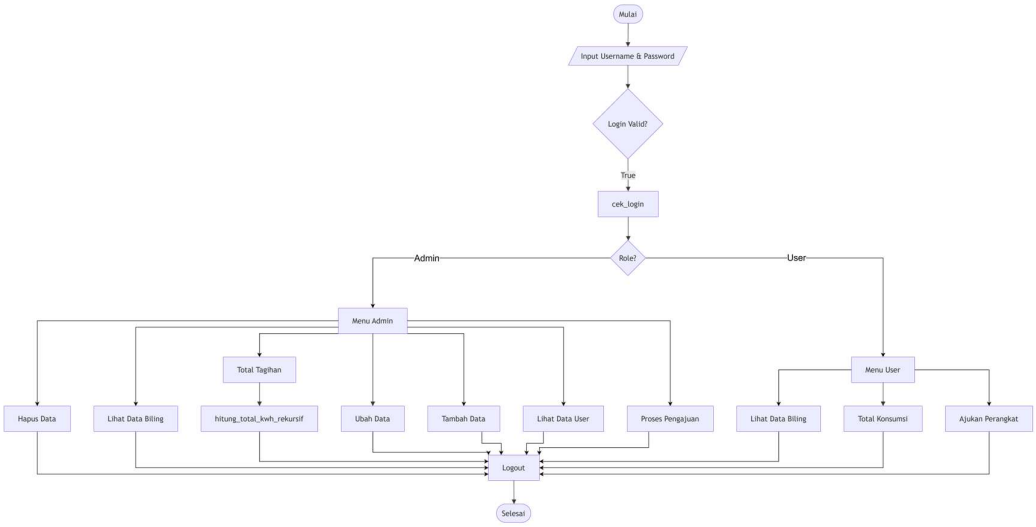
LAPORAN PRAKTIKUM
POSTTEST 8
ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:
Adzkia Firdalina (2609106033)
Kelas (A2 ‘26)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart



Penjelasan Flowchart

Flowchart WattWise menggambarkan alur kerja program mulai dari proses login dan registrasi pengguna hingga pengelolaan serta perhitungan penggunaan listrik. Program terlebih dahulu memvalidasi username dan password, kemudian menentukan menu berdasarkan peran pengguna yang berhasil login.

Admin memiliki akses untuk melihat, menambahkan, mengubah, dan menghapus data penggunaan listrik, melihat daftar pengguna, serta menghitung total konsumsi listrik. Sementara itu, user dapat melihat data penggunaan listrik dan mengetahui estimasi konsumsi serta biaya listrik.

Dalam proses pengolahan data, program menghitung konsumsi listrik setiap perangkat berdasarkan daya dan durasi pemakaian. Hasil perhitungan kemudian dijumlahkan untuk memperoleh total konsumsi listrik dan dikonversi menjadi estimasi biaya berdasarkan tarif listrik yang telah ditentukan.

2. Deskripsi Singkat Program

WattWise merupakan program berbasis Python yang dirancang untuk membantu pengguna memantau penggunaan energi listrik dan memperkirakan biaya pemakaian listrik berdasarkan daya perangkat serta durasi penggunaannya. Program ini menerapkan sistem login dengan dua peran pengguna, yaitu admin dan user, yang memiliki hak akses berbeda.

Fitur yang tersedia meliputi registrasi pengguna, autentikasi login, pengelolaan data penggunaan listrik, penampilan daftar perangkat, serta perhitungan konsumsi listrik dalam satuan kilowatt-hour (kWh). Program juga menyediakan estimasi biaya listrik berdasarkan tarif Rp1.444,70 per kWh, termasuk perkiraan biaya bulanan untuk pengguna.

Dalam implementasinya, WattWise menerapkan beberapa konsep dasar pemrograman, seperti variabel, list, perulangan, percabangan, fungsi bawaan Python, dan pengolahan data numerik. Program ini diharapkan dapat membantu pengguna memahami pola konsumsi energi listrik sekaligus menerapkan konsep algoritma pemrograman dasar dalam suatu aplikasi sederhana.

3. Source Code

```
import os
import datetime
import sys

try:
    from prettytable import PrettyTable
    from colorama import Fore, init
except ImportError:
    print("[!] Library belum terinstall.")
    print("    Jalankan dulu: pip install prettytable colorama")
    sys.exit(1)

init(autoreset=True)


# ============================================================
# KONFIGURASI & DATA
# ============================================================
TARIF = 1444.7

USERS = [
    ["admin", "admin123", "admin"],
    ["kiki", "kiki123", "user"]
]

DATA_LISTRIK = [
    [1, "Lampu", 10, 5, "12-10-2026"],
    [2, "Kipas Angin", 50, 8, "12-10-2026"],
    [3, "Kulkas", 100, 24, "12-10-2026"]
]

PENGAJUAN = []


# ============================================================
# UTIL
# ============================================================
def bersihkan_layar():
    os.system("cls" if os.name == "nt" else "clear")


def format_durasi(durasi):
    return int(durasi) if float(durasi) == int(durasi) else durasi

def validasi_tanggal(tanggal):
    bagian = tanggal.split("-")
    if len(bagian) != 3:
        raise ValueError("Format tanggal harus dd-mm-yyyy")

    try:
```

```
        dd = int(bagian[0])
        mm = int(bagian[1])
        yy = int(bagian[2])
    except ValueError:
        raise ValueError("Tanggal harus berisi angka semua")

    if dd < 1 or dd > 31 or mm < 1 or mm > 12 or yy < 2000:
        raise ValueError("Tanggal tidak valid")

    return True


def validasi_nama(nama):
    nama = nama.strip()
    if nama == "":
        raise ValueError("Nama perangkat tidak boleh kosong")
    return nama


def validasi_daya(daya):
    try:
        nilai = int(daya)
    except (ValueError, TypeError):
        raise ValueError("Daya harus angka")

    if nilai <= 0:
        raise ValueError("Daya harus lebih dari 0")

    return nilai


def validasi_durasi(durasi):
    try:
        nilai = float(durasi)
    except (ValueError, TypeError):
        raise ValueError("Durasi harus angka")

    if nilai <= 0:
        raise ValueError("Durasi harus lebih dari 0")

    if nilai > 24:
        raise ValueError("Durasi sehari tidak mungkin lebih dari 24 jam")

    return nilai

def cek_login(username, password):
    for x in USERS:
        if x[0] == username and x[1] == password:
            return x[2]
    return None


def username_terdaftar(username):
    for x in USERS:
```

```
        if x[0] == username:
            return True
    return False


def registrasi_user(username, password):
    username = username.strip().lower()
    password = password.strip().lower()

    if username == "":
        raise ValueError("Username tidak boleh kosong")

    if password == "":
        raise ValueError("Password tidak boleh kosong")

    if username_terdaftar(username):
        raise ValueError("Username sudah dipakai, cari yang lain")

    USERS.append([username, password, "user"])
    return username

def hitung_kwh(daya, durasi):
    return float(daya) * float(durasi) / 1000


def hitung_tagihan(kwh):
    return kwh * TARIF


def hitung_total_kwh_rekursif(data, index=0):
    if index == len(data):
        return 0

    kwh = hitung_kwh(data[index][2], data[index][3])
    return kwh + hitung_total_kwh_rekursif(data, index + 1)


def hitung_total_kwh(data):
    return hitung_total_kwh_rekursif(data)

def get_id_listrik_baru():
    if len(DATA_LISTRIK) == 0:
        return 1
    return DATA_LISTRIK[-1][0] + 1


def get_id_pengajuan_baru():
    if len(PENGAJUAN) == 0:
        return 1
    return PENGAJUAN[-1][0] + 1


def cari_index_listrik(id_cari):
    for i in range(len(DATA_LISTRIK)):
```

```
        if DATA_LISTRIK[i][0] == id_cari:
            return i
    return -1


def cari_index_pengajuan(id_cari):
    for i in range(len(PENGAJUAN)):
        if PENGAJUAN[i][0] == id_cari:
            return i
    return -1


def tambah_data_listrik(nama, daya, durasi, tanggal):
    id_baru = get_id_listrik_baru()
    DATA_LISTRIK.append([id_baru, nama, daya, durasi, tanggal])
    return id_baru


def ubah_data_listrik(posisi, nama, daya, durasi, tanggal):
    DATA_LISTRIK[posisi][1] = nama
    DATA_LISTRIK[posisi][2] = daya
    DATA_LISTRIK[posisi][3] = durasi
    DATA_LISTRIK[posisi][4] = tanggal


def hapus_data_listrik(posisi):
    DATA_LISTRIK.pop(posisi)


def tambah_pengajuan(username, nama, daya, durasi, tanggal):
    id_baru = get_id_pengajuan_baru()
    PENGAJUAN.append([id_baru, username, nama, daya, durasi, tanggal])
    return id_baru


def hapus_pengajuan(posisi):
    PENGAJUAN.pop(posisi)

def tampilkan_menu(role):
    print(Fore.CYAN + "=" * 50)
    if role == "admin":
        print("1. View Data Biling    4. Hapus Data Biling")
        print("2. Tambah Data Biling  5. Lihat Informasi User")
        print("3. Ubah Data Biling    6. Total Tagihan")
        print("7. Lihat Pengajuan User")
    else:
        print("1. View Data Biling")
        print("2. Total Konsumsi Saya")
        print("3. Ajukan Penambahan Perangkat")
    print("0. Logout")
    print("=" * 50 + Fore.RESET)


def tampilkan_data_biling():
```

```
    bersihkan_layar()
    print(Fore.CYAN + "=" * 70)
    print("            DAFTAR PENGGUNAAN LISTRIK")
    print("=" * 70 + Fore.RESET)

    if len(DATA_LISTRIK) == 0:
        print(Fore.YELLOW + "Belum ada data" + Fore.RESET)
        input("Tekan Enter Untuk Melanjutkan...")
        return

    table = PrettyTable()
    table.field_names = [
        "ID", "Perangkat", "Daya (W)",
        "Durasi (jam)", "Tanggal", "kWh", "Biaya"
    ]
    table.align["Perangkat"] = "l"

    total = hitung_total_kwh(DATA_LISTRIK)

    for d in DATA_LISTRIK:
        kwh = hitung_kwh(d[2], d[3])
        biaya = hitung_tagihan(kwh)
        table.add_row([
            d[0], d[1], d[2], format_durasi(d[3]), d[4],
            f"{kwh:.3f}", f"Rp {biaya:,.2f}"
        ])

    total_biaya = hitung_tagihan(total)
    table.add_row([
        "", "TOTAL", "", "", "",
        f"{total:.3f}", f"Rp {total_biaya:,.2f}"
    ])

    print(table)
    input("Tekan Enter Untuk Melanjutkan...")


def tampilkan_daftar_user():
    bersihkan_layar()
    print(Fore.CYAN + "=== DAFTAR USER ===" + Fore.RESET)

    table = PrettyTable()
    table.field_names = ["No", "Username", "Role"]
    table.align["Username"] = "l"

    for i, x in enumerate(USERS, start=1):
        table.add_row([i, x[0], x[2]])

    print(table)
    input("Tekan Enter untuk melanjutkan...")


def tampilkan_total_penggunaan():
    bersihkan_layar()
```

```
    print(Fore.CYAN + "=== TOTAL PENGGUNAAN ===" + Fore.RESET)

    if len(DATA_LISTRIK) == 0:
        print(Fore.YELLOW + "Belum ada data" + Fore.RESET)
        input("Tekan Enter untuk melanjutkan...")
        return

    table = PrettyTable()
    table.field_names = ["Perangkat", "kWh", "Biaya"]
    table.align["Perangkat"] = "l"

    total = 0
    for d in DATA_LISTRIK:
        kwh = hitung_kwh(d[2], d[3])
        total += kwh
        table.add_row([
            d[1],
            f"{kwh:.3f}",
            f"Rp {hitung_tagihan(kwh):,.2f}"
        ])

    table.add_row([
        "TOTAL",
        f"{total:.3f}",
        f"Rp {hitung_tagihan(total):,.2f}"
    ])

    print(table)
    input("Tekan Enter untuk melanjutkan...")


def tampilkan_pengajuan():
    """Tampilkan tabel pengajuan. Return True jika ada data."""
    bersihkan_layar()
    print(Fore.CYAN + "=== PENGAJUAN DARI USER ===" + Fore.RESET)

    if not PENGAJUAN:
        print(Fore.YELLOW + "Belum ada pengajuan nih" + Fore.RESET)
        input("Tekan Enter Untuk Melanjutkan...")
        return False

    table = PrettyTable()
    table.field_names = [
        "ID", "User", "Perangkat",
        "Daya (W)", "Durasi (jam)", "Tanggal"
    ]
    table.align["User"] = "l"
    table.align["Perangkat"] = "l"

    for p in PENGAJUAN:
        table.add_row([
            p[0], p[1], p[2], p[3], format_durasi(p[4]), p[5]
        ])
```

```
    print(table)
    input("Tekan Enter untuk lanjut memproses...")
    return True


def tampilkan_total_user():
    bersihkan_layar()
    print(Fore.CYAN + "=== TOTAL PENGGUNAAN ANDA ===" + Fore.RESET)

    total = hitung_total_kwh(DATA_LISTRIK)

    table = PrettyTable()
    table.field_names = ["Keterangan", "Nilai"]
    table.add_row(["Harian", f"{total:.3f} kWh"])
    table.add_row(["Biaya", f"Rp {hitung_tagihan(total):,.2f}"])
    table.add_row([
        "Bulanan",
        f"Rp {hitung_tagihan(total) * 30:,.2f}"
    ])

    print(table)
    input("Tekan Enter...")

def tambah_data_admin():
    bersihkan_layar()
    print("=== TAMBAH DATA ===")
    print("Ketik 0 di nama perangkat buat batal\n")

    nama = input("Nama perangkat      : ").strip()
    if nama == "0":
        print("Gajadi nambah data")
        input("Tekan Enter untuk melanjutkan...")
        return

    daya = input("Daya (Watt)         : ").strip()
    durasi = input("Durasi (jam)        : ").strip()
    tanggal = input("Tanggal (dd-mm-yyyy): ").strip()

    nama = validasi_nama(nama)
    daya_int = validasi_daya(daya)
    durasi_float = validasi_durasi(durasi)
    validasi_tanggal(tanggal)

    for d in DATA_LISTRIK:
        if d[1].lower() == nama.lower() and d[4] == tanggal:
            raise ValueError("Data ini udah ada")

    id_baru = tambah_data_listrik(nama, daya_int, durasi_float, tanggal)
    print(f"[+] Data berhasil ditambah, ID nya {id_baru}")
    input("Tekan Enter untuk melanjutkan...")


def ubah_data_admin():
    bersihkan_layar()
```

```
    print("=== UBAH DATA ===")

    for d in DATA_LISTRIK:
        print(f"{d[0]}. {d[1]} - {d[2]}W - {format_durasi(d[3])}j - {d[4]}")

    print("")
    idx_input = input("ID yang mau diubah (0 buat batal): ").strip()

    if idx_input == "0":
        print("Gajadi ubah")
        input("Tekan Enter untuk melanjutkan...")
        return

    try:
        idx = int(idx_input)
    except ValueError:
        raise ValueError("ID harus angka")

    posisi = cari_index_listrik(idx)
    if posisi == -1:
        raise ValueError("ID gak ketemu")

    nama = input("Nama baru    : ").strip()
    daya = input("Daya baru    : ").strip()
    durasi = input("Durasi baru  : ").strip()
    tanggal = input("Tanggal baru : ").strip()

    nama = validasi_nama(nama)
    daya_int = validasi_daya(daya)
    durasi_float = validasi_durasi(durasi)
    validasi_tanggal(tanggal)

    ubah_data_listrik(posisi, nama, daya_int, durasi_float, tanggal)
    print("[+] Data udah diubah")
    input("Tekan Enter untuk melanjutkan...")


def hapus_data_admin():
    bersihkan_layar()
    print("=== HAPUS DATA ===")

    for d in DATA_LISTRIK:
        print(f"{d[0]}. {d[1]} - {d[4]}")

    print("")
    idx_input = input("ID yang mau dihapus (0 buat batal): ").strip()

    if idx_input == "0":
        print("Gajadi hapus")
        input("Tekan Enter untuk melanjutkan...")
        return

    try:
        idx = int(idx_input)
```

```
    except ValueError:
        raise ValueError("ID harus angka")

    posisi = cari_index_listrik(idx)
    if posisi == -1:
        raise ValueError("Sayang banget ID mu gak ketemu")

    tanya  =  input(f"Yakin  mau  hapus  {DATA_LISTRIK[posisi][1]}?  (y/n):
").lower()

    if tanya == "y":
        hapus_data_listrik(posisi)
        print("[+] Data udah dihapus")
    else:
        print("Gajadi hapus")

    input("Tekan Enter untuk melanjutkan...")


def proses_pengajuan_admin():
    ada = tampilkan_pengajuan()
    if not ada:
        return

    id_input = input(
        "Masukkan ID pengajuan yang mau diproses (0 buat batal): "
    ).strip()

    if id_input == "0":
        print("Gajadi proses")
        input("Tekan Enter Untuk Melanjutkan...")
        return

    try:
        id_pil = int(id_input)
    except ValueError:
        raise ValueError("ID harus angka")

    posisi = cari_index_pengajuan(id_pil)
    if posisi == -1:
        raise ValueError("ID pengajuan gak ketemu")

    print("")
    print(f"Pengajuan dari : {PENGAJUAN[posisi][1]}")
    print(f"Perangkat      : {PENGAJUAN[posisi][2]}")
    print(f"Daya           : {PENGAJUAN[posisi][3]} W")
    print(f"Durasi         : {format_durasi(PENGAJUAN[posisi][4])} jam")
    print(f"Tanggal        : {PENGAJUAN[posisi][5]}")
    print("")

    aksi = input("Mau di-ACC atau ditolak? (acc/tolak): ").strip().lower()

    if aksi == "acc":
        dobel = False
```

```
        for d in DATA_LISTRIK:
            if (
                d[1].lower() == PENGAJUAN[posisi][2].lower()
                and d[4] == PENGAJUAN[posisi][5]
            ):
                dobel = True

        if dobel:
            print("Perangkat ini udah ada di data biling")
        else:
            tambah_data_listrik(
                PENGAJUAN[posisi][2],
                PENGAJUAN[posisi][3],
                PENGAJUAN[posisi][4],
                PENGAJUAN[posisi][5]
            )
            hapus_pengajuan(posisi)
            print("Pengajuan di-ACC, data udah masuk ke biling")

    elif aksi == "tolak":
        hapus_pengajuan(posisi)
        print("Pengajuan ditolak")

    else:
        print("Pilihan gak ada, coba lagi")

    input("Tekan Enter Untuk Melanjutkan...")

def ajukan_perangkat(username):
    bersihkan_layar()
    print("=== AJUKAN PENAMBAHAN PERANGKAT ===")
    print("[i] Isi datanya, admin bakal tinjau dulu\n")

    nama = input("Mau nambah perangkat apa? ").strip()
    daya = input("Berapa watt? ").strip()
    durasi = input("Berapa jam pemakaian per hari? ").strip()

    hari_ini = datetime.date.today()
    tanggal = f"{hari_ini.day:02d}-{hari_ini.month:02d}-{hari_ini.year}"

    nama = validasi_nama(nama)
    daya_int = validasi_daya(daya)
    durasi_float = validasi_durasi(durasi)

    for d in DATA_LISTRIK:
        if d[1].lower() == nama.lower():
            raise ValueError("Perangkat ini udah ada di data biling")

    for p in PENGAJUAN:
        if p[2].lower() == nama.lower():
            raise ValueError("Kamu udah pernah ngajuin perangkat ini")

    id_peng = tambah_pengajuan(
        username, nama, daya_int, durasi_float, tanggal
```

```
    )

    print("")
    print(" Pengajuan anda berhasil dikirim!")
    print(f"    ID        : {id_peng}")
    print(f"    Perangkat : {nama}")
    print(f"    Watt      : {daya_int} W")
    print(f"    Jam       : {format_durasi(durasi_float)} jam")
    print(f"    Tanggal   : {tanggal}")
    print("")
    print("    Tunggu di ACC admin ya")
    input("Tekan Enter untuk melanjutkan...")

def main():
    while True:
        bersihkan_layar()
        print("=" * 50)
        print("        WELCOME TO WATTWISE")
        print("   Monitor Ur Electricity Usage")
        print("=" * 50)

        username = input("Username : ").strip().lower()
        password = input("Password : ").strip().lower()

        if not username_terdaftar(username):
            print("")
            print(f"Username {username} anda belum terdaftar.")
            jawab = input("Registrasi sekarang? (y/n): ").strip().lower()

            if jawab == "y":
                try:
                    nu = input("Username baru : ").strip().lower()
                    np = input("Password baru : ").strip().lower()
                    nu = registrasi_user(nu, np)
                    print(f"Berhasil daftar! Silakan login pake username:
{nu}")
                except ValueError as e:
                    print(f"Gagal registrasi: {e}")

                input("Tekan Enter untuk melanjutkan...")
            else:
                print("")
                print("Yaudah keluar dulu ya. Terima kasih!")
                break

            continue

        role = cek_login(username, password)

        if role is None:
            print("")
            print("Password salah!")
            input("Tekan Enter untuk melanjutkan...")
            continue
```

```
        print("")
        print(f"Login sebagai {role.upper()}. Halo, {username}!")
        input("Tekan Enter untuk melanjutkan...")

        while True:
            bersihkan_layar()
            print("=" * 50)
            print(f"   MENU {role.upper()} - {username}")
            tampilkan_menu(role)

            pilih = input("Pilih: ").strip()

            try:
                if pilih == "1":
                    tampilkan_data_biling()

                elif pilih == "2" and role == "admin":
                    tambah_data_admin()

                elif pilih == "3" and role == "admin":
                    ubah_data_admin()

                elif pilih == "4" and role == "admin":
                    hapus_data_admin()

                elif pilih == "5" and role == "admin":
                    tampilkan_daftar_user()

                elif pilih == "6" and role == "admin":
                    tampilkan_total_penggunaan()

                elif pilih == "7" and role == "admin":
                    proses_pengajuan_admin()

                elif pilih == "2" and role == "user":
                    tampilkan_total_user()

                elif pilih == "3" and role == "user":
                    ajukan_perangkat(username)

                elif pilih == "0":
                    print("")
                    print("Yah, Logout dulu ya")
                    input("Tekan Enter untuk melanjutkan...")
                    break

                else:
                    print("Yhaaaa Pilihannya gak ada, silahkan coba lagi")
                    input("Tekan Enter untuk melanjutkan...")

            except ValueError as e:
                print(f"[!] Error: {e}")
                input("Tekan Enter untuk melanjutkan...")
```

```
            except Exception as e:
                print(f"[!] Terjadi kesalahan: {e}")
                input("Tekan Enter untuk melanjutkan...")


if __name__ == "__main__":
    main()
import os
import datetime
import sys

try:
    from prettytable import PrettyTable
    from colorama import Fore, init
except ImportError:
    print("[!] Library belum terinstall.")
    print("    Jalankan dulu: pip install prettytable colorama")
    sys.exit(1)

init(autoreset=True)


# ============================================================
# KONFIGURASI & DATA
# ============================================================
TARIF = 1444.7

USERS = [
    ["admin", "admin123", "admin"],
    ["kiki", "kiki123", "user"]
]

DATA_LISTRIK = [
    [1, "Lampu", 10, 5, "12-10-2026"],
    [2, "Kipas Angin", 50, 8, "12-10-2026"],
    [3, "Kulkas", 100, 24, "12-10-2026"]
]

PENGAJUAN = []


# ============================================================
# UTIL
# ============================================================
def bersihkan_layar():
    os.system("cls" if os.name == "nt" else "clear")


def format_durasi(durasi):
    return int(durasi) if float(durasi) == int(durasi) else durasi

def validasi_tanggal(tanggal):
    bagian = tanggal.split("-")
```

```
    if len(bagian) != 3:
        raise ValueError("Format tanggal harus dd-mm-yyyy")

    try:
        dd = int(bagian[0])
        mm = int(bagian[1])
        yy = int(bagian[2])
    except ValueError:
        raise ValueError("Tanggal harus berisi angka semua")

    if dd < 1 or dd > 31 or mm < 1 or mm > 12 or yy < 2000:
        raise ValueError("Tanggal tidak valid")

    return True


def validasi_nama(nama):
    nama = nama.strip()
    if nama == "":
        raise ValueError("Nama perangkat tidak boleh kosong")
    return nama


def validasi_daya(daya):
    try:
        nilai = int(daya)
    except (ValueError, TypeError):
        raise ValueError("Daya harus angka")

    if nilai <= 0:
        raise ValueError("Daya harus lebih dari 0")

    return nilai


def validasi_durasi(durasi):
    try:
        nilai = float(durasi)
    except (ValueError, TypeError):
        raise ValueError("Durasi harus angka")

    if nilai <= 0:
        raise ValueError("Durasi harus lebih dari 0")

    if nilai > 24:
        raise ValueError("Durasi sehari tidak mungkin lebih dari 24 jam")

    return nilai

def cek_login(username, password):
    for x in USERS:
        if x[0] == username and x[1] == password:
            return x[2]
    return None
```

```
def username_terdaftar(username):
    for x in USERS:
        if x[0] == username:
            return True
    return False


def registrasi_user(username, password):
    username = username.strip().lower()
    password = password.strip().lower()

    if username == "":
        raise ValueError("Username tidak boleh kosong")

    if password == "":
        raise ValueError("Password tidak boleh kosong")

    if username_terdaftar(username):
        raise ValueError("Username sudah dipakai, cari yang lain")

    USERS.append([username, password, "user"])
    return username

def hitung_kwh(daya, durasi):
    return float(daya) * float(durasi) / 1000


def hitung_tagihan(kwh):
    return kwh * TARIF


def hitung_total_kwh_rekursif(data, index=0):
    if index == len(data):
        return 0

    kwh = hitung_kwh(data[index][2], data[index][3])
    return kwh + hitung_total_kwh_rekursif(data, index + 1)


def hitung_total_kwh(data):
    return hitung_total_kwh_rekursif(data)

def get_id_listrik_baru():
    if len(DATA_LISTRIK) == 0:
        return 1
    return DATA_LISTRIK[-1][0] + 1


def get_id_pengajuan_baru():
    if len(PENGAJUAN) == 0:
        return 1
    return PENGAJUAN[-1][0] + 1
```

```
def cari_index_listrik(id_cari):
    for i in range(len(DATA_LISTRIK)):
        if DATA_LISTRIK[i][0] == id_cari:
            return i
    return -1

def cari_index_pengajuan(id_cari):
    for i in range(len(PENGAJUAN)):
        if PENGAJUAN[i][0] == id_cari:
            return i
    return -1

def tambah_data_listrik(nama, daya, durasi, tanggal):
    id_baru = get_id_listrik_baru()
    DATA_LISTRIK.append([id_baru, nama, daya, durasi, tanggal])
    return id_baru

def ubah_data_listrik(posisi, nama, daya, durasi, tanggal):
    DATA_LISTRIK[posisi][1] = nama
    DATA_LISTRIK[posisi][2] = daya
    DATA_LISTRIK[posisi][3] = durasi
    DATA_LISTRIK[posisi][4] = tanggal

def hapus_data_listrik(posisi):
    DATA_LISTRIK.pop(posisi)

def tambah_pengajuan(username, nama, daya, durasi, tanggal):
    id_baru = get_id_pengajuan_baru()
    PENGAJUAN.append([id_baru, username, nama, daya, durasi, tanggal])
    return id_baru

def hapus_pengajuan(posisi):
    PENGAJUAN.pop(posisi)

def tampilkan_menu(role):
    print(Fore.CYAN + "=" * 50)
    if role == "admin":
        print("1. View Data Biling    4. Hapus Data Biling")
        print("2. Tambah Data Biling  5. Lihat Informasi User")
        print("3. Ubah Data Biling    6. Total Tagihan")
        print("7. Lihat Pengajuan User")
    else:
        print("1. View Data Biling")
        print("2. Total Konsumsi Saya")
        print("3. Ajukan Penambahan Perangkat")
    print("0. Logout")
```

```
    print("=" * 50 + Fore.RESET)


def tampilkan_data_biling():
    bersihkan_layar()
    print(Fore.CYAN + "=" * 70)
    print("            DAFTAR PENGGUNAAN LISTRIK")
    print("=" * 70 + Fore.RESET)

    if len(DATA_LISTRIK) == 0:
        print(Fore.YELLOW + "Belum ada data" + Fore.RESET)
        input("Tekan Enter Untuk Melanjutkan...")
        return

    table = PrettyTable()
    table.field_names = [
        "ID", "Perangkat", "Daya (W)",
        "Durasi (jam)", "Tanggal", "kWh", "Biaya"
    ]
    table.align["Perangkat"] = "l"

    total = hitung_total_kwh(DATA_LISTRIK)

    for d in DATA_LISTRIK:
        kwh = hitung_kwh(d[2], d[3])
        biaya = hitung_tagihan(kwh)
        table.add_row([
            d[0], d[1], d[2], format_durasi(d[3]), d[4],
            f"{kwh:.3f}", f"Rp {biaya:,.2f}"
        ])

    total_biaya = hitung_tagihan(total)
    table.add_row([
        "", "TOTAL", "", "", "",
        f"{total:.3f}", f"Rp {total_biaya:,.2f}"
    ])

    print(table)
    input("Tekan Enter Untuk Melanjutkan...")


def tampilkan_daftar_user():
    bersihkan_layar()
    print(Fore.CYAN + "=== DAFTAR USER ===" + Fore.RESET)

    table = PrettyTable()
    table.field_names = ["No", "Username", "Role"]
    table.align["Username"] = "l"

    for i, x in enumerate(USERS, start=1):
        table.add_row([i, x[0], x[2]])

    print(table)
    input("Tekan Enter untuk melanjutkan...")
```

```
def tampilkan_total_penggunaan():
    bersihkan_layar()
    print(Fore.CYAN + "=== TOTAL PENGGUNAAN ===" + Fore.RESET)

    if len(DATA_LISTRIK) == 0:
        print(Fore.YELLOW + "Belum ada data" + Fore.RESET)
        input("Tekan Enter untuk melanjutkan...")
        return

    table = PrettyTable()
    table.field_names = ["Perangkat", "kWh", "Biaya"]
    table.align["Perangkat"] = "l"

    total = 0
    for d in DATA_LISTRIK:
        kwh = hitung_kwh(d[2], d[3])
        total += kwh
        table.add_row([
            d[1],
            f"{kwh:.3f}",
            f"Rp {hitung_tagihan(kwh):,.2f}"
        ])

    table.add_row([
        "TOTAL",
        f"{total:.3f}",
        f"Rp {hitung_tagihan(total):,.2f}"
    ])

    print(table)
    input("Tekan Enter untuk melanjutkan...")


def tampilkan_pengajuan():
    """Tampilkan tabel pengajuan. Return True jika ada data."""
    bersihkan_layar()
    print(Fore.CYAN + "=== PENGAJUAN DARI USER ===" + Fore.RESET)

    if not PENGAJUAN:
        print(Fore.YELLOW + "Belum ada pengajuan nih" + Fore.RESET)
        input("Tekan Enter Untuk Melanjutkan...")
        return False

    table = PrettyTable()
    table.field_names = [
        "ID", "User", "Perangkat",
        "Daya (W)", "Durasi (jam)", "Tanggal"
    ]
    table.align["User"] = "l"
    table.align["Perangkat"] = "l"

    for p in PENGAJUAN:
```

```
        table.add_row([
            p[0], p[1], p[2], p[3], format_durasi(p[4]), p[5]
        ])

    print(table)
    input("Tekan Enter untuk lanjut memproses...")
    return True


def tampilkan_total_user():
    bersihkan_layar()
    print(Fore.CYAN + "=== TOTAL PENGGUNAAN ANDA ===" + Fore.RESET)

    total = hitung_total_kwh(DATA_LISTRIK)

    table = PrettyTable()
    table.field_names = ["Keterangan", "Nilai"]
    table.add_row(["Harian", f"{total:.3f} kWh"])
    table.add_row(["Biaya", f"Rp {hitung_tagihan(total):,.2f}"])
    table.add_row([
        "Bulanan",
        f"Rp {hitung_tagihan(total) * 30:,.2f}"
    ])

    print(table)
    input("Tekan Enter...")

def tambah_data_admin():
    bersihkan_layar()
    print("=== TAMBAH DATA ===")
    print("Ketik 0 di nama perangkat buat batal\n")

    nama = input("Nama perangkat      : ").strip()
    if nama == "0":
        print("Gajadi nambah data")
        input("Tekan Enter untuk melanjutkan...")
        return

    daya = input("Daya (Watt)         : ").strip()
    durasi = input("Durasi (jam)        : ").strip()
    tanggal = input("Tanggal (dd-mm-yyyy): ").strip()

    nama = validasi_nama(nama)
    daya_int = validasi_daya(daya)
    durasi_float = validasi_durasi(durasi)
    validasi_tanggal(tanggal)

    for d in DATA_LISTRIK:
        if d[1].lower() == nama.lower() and d[4] == tanggal:
            raise ValueError("Data ini udah ada")

    id_baru = tambah_data_listrik(nama, daya_int, durasi_float, tanggal)
    print(f"[+] Data berhasil ditambah, ID nya {id_baru}")
    input("Tekan Enter untuk melanjutkan...")
```

```
def ubah_data_admin():
    bersihkan_layar()
    print("=== UBAH DATA ===")

    for d in DATA_LISTRIK:
        print(f"{d[0]}. {d[1]} - {d[2]}W - {format_durasi(d[3])}j - {d[4]}")

    print("")
    idx_input = input("ID yang mau diubah (0 buat batal): ").strip()

    if idx_input == "0":
        print("Gajadi ubah")
        input("Tekan Enter untuk melanjutkan...")
        return

    try:
        idx = int(idx_input)
    except ValueError:
        raise ValueError("ID harus angka")

    posisi = cari_index_listrik(idx)
    if posisi == -1:
        raise ValueError("ID gak ketemu")

    nama = input("Nama baru    : ").strip()
    daya = input("Daya baru    : ").strip()
    durasi = input("Durasi baru  : ").strip()
    tanggal = input("Tanggal baru : ").strip()

    nama = validasi_nama(nama)
    daya_int = validasi_daya(daya)
    durasi_float = validasi_durasi(durasi)
    validasi_tanggal(tanggal)

    ubah_data_listrik(posisi, nama, daya_int, durasi_float, tanggal)
    print("[+] Data udah diubah")
    input("Tekan Enter untuk melanjutkan...")


def hapus_data_admin():
    bersihkan_layar()
    print("=== HAPUS DATA ===")

    for d in DATA_LISTRIK:
        print(f"{d[0]}. {d[1]} - {d[4]}")

    print("")
    idx_input = input("ID yang mau dihapus (0 buat batal): ").strip()

    if idx_input == "0":
        print("Gajadi hapus")
        input("Tekan Enter untuk melanjutkan...")
```

```
        return

    try:
        idx = int(idx_input)
    except ValueError:
        raise ValueError("ID harus angka")

    posisi = cari_index_listrik(idx)
    if posisi == -1:
        raise ValueError("Sayang banget ID mu gak ketemu")

    tanya  =  input(f"Yakin  mau  hapus  {DATA_LISTRIK[posisi][1]}?  (y/n):
").lower()

    if tanya == "y":
        hapus_data_listrik(posisi)
        print("[+] Data udah dihapus")
    else:
        print("Gajadi hapus")

    input("Tekan Enter untuk melanjutkan...")


def proses_pengajuan_admin():
    ada = tampilkan_pengajuan()
    if not ada:
        return

    id_input = input(
        "Masukkan ID pengajuan yang mau diproses (0 buat batal): "
    ).strip()

    if id_input == "0":
        print("Gajadi proses")
        input("Tekan Enter Untuk Melanjutkan...")
        return

    try:
        id_pil = int(id_input)
    except ValueError:
        raise ValueError("ID harus angka")

    posisi = cari_index_pengajuan(id_pil)
    if posisi == -1:
        raise ValueError("ID pengajuan gak ketemu")

    print("")
    print(f"Pengajuan dari : {PENGAJUAN[posisi][1]}")
    print(f"Perangkat      : {PENGAJUAN[posisi][2]}")
    print(f"Daya           : {PENGAJUAN[posisi][3]} W")
    print(f"Durasi         : {format_durasi(PENGAJUAN[posisi][4])} jam")
    print(f"Tanggal        : {PENGAJUAN[posisi][5]}")
    print("")
```

```
    aksi = input("Mau di-ACC atau ditolak? (acc/tolak): ").strip().lower()

    if aksi == "acc":
        dobel = False
        for d in DATA_LISTRIK:
            if (
                d[1].lower() == PENGAJUAN[posisi][2].lower()
                and d[4] == PENGAJUAN[posisi][5]
            ):
                dobel = True

        if dobel:
            print("Perangkat ini udah ada di data biling")
        else:
            tambah_data_listrik(
                PENGAJUAN[posisi][2],
                PENGAJUAN[posisi][3],
                PENGAJUAN[posisi][4],
                PENGAJUAN[posisi][5]
            )
            hapus_pengajuan(posisi)
            print("Pengajuan di-ACC, data udah masuk ke biling")

    elif aksi == "tolak":
        hapus_pengajuan(posisi)
        print("Pengajuan ditolak")

    else:
        print("Pilihan gak ada, coba lagi")

    input("Tekan Enter Untuk Melanjutkan...")

def ajukan_perangkat(username):
    bersihkan_layar()
    print("=== AJUKAN PENAMBAHAN PERANGKAT ===")
    print("[i] Isi datanya, admin bakal tinjau dulu\n")

    nama = input("Mau nambah perangkat apa? ").strip()
    daya = input("Berapa watt? ").strip()
    durasi = input("Berapa jam pemakaian per hari? ").strip()

    hari_ini = datetime.date.today()
    tanggal = f"{hari_ini.day:02d}-{hari_ini.month:02d}-{hari_ini.year}"

    nama = validasi_nama(nama)
    daya_int = validasi_daya(daya)
    durasi_float = validasi_durasi(durasi)

    for d in DATA_LISTRIK:
        if d[1].lower() == nama.lower():
            raise ValueError("Perangkat ini udah ada di data biling")

    for p in PENGAJUAN:
        if p[2].lower() == nama.lower():
```

```
            raise ValueError("Kamu udah pernah ngajuin perangkat ini")

    id_peng = tambah_pengajuan(
        username, nama, daya_int, durasi_float, tanggal
    )

    print("")
    print(" Pengajuan anda berhasil dikirim!")
    print(f"    ID        : {id_peng}")
    print(f"    Perangkat : {nama}")
    print(f"    Watt      : {daya_int} W")
    print(f"    Jam       : {format_durasi(durasi_float)} jam")
    print(f"    Tanggal   : {tanggal}")
    print("")
    print("    Tunggu di ACC admin ya")
    input("Tekan Enter untuk melanjutkan...")

def main():
    while True:
        bersihkan_layar()
        print("=" * 50)
        print("        WELCOME TO WATTWISE")
        print("   Monitor Ur Electricity Usage")
        print("=" * 50)

        username = input("Username : ").strip().lower()
        password = input("Password : ").strip().lower()

        if not username_terdaftar(username):
            print("")
            print(f"Username {username} anda belum terdaftar.")
            jawab = input("Registrasi sekarang? (y/n): ").strip().lower()

            if jawab == "y":
                try:
                    nu = input("Username baru : ").strip().lower()
                    np = input("Password baru : ").strip().lower()
                    nu = registrasi_user(nu, np)
                    print(f"Berhasil daftar! Silakan login pake username:
{nu}")
                except ValueError as e:
                    print(f"Gagal registrasi: {e}")

                input("Tekan Enter untuk melanjutkan...")
            else:
                print("")
                print("Yaudah keluar dulu ya. Terima kasih!")
                break

            continue

        role = cek_login(username, password)

        if role is None:
```

```
            print("")
            print("Password salah!")
            input("Tekan Enter untuk melanjutkan...")
            continue

        print("")
        print(f"Login sebagai {role.upper()}. Halo, {username}!")
        input("Tekan Enter untuk melanjutkan...")

        while True:
            bersihkan_layar()
            print("=" * 50)
            print(f"   MENU {role.upper()} - {username}")
            tampilkan_menu(role)

            pilih = input("Pilih: ").strip()

            try:
                if pilih == "1":
                    tampilkan_data_biling()

                elif pilih == "2" and role == "admin":
                    tambah_data_admin()

                elif pilih == "3" and role == "admin":
                    ubah_data_admin()

                elif pilih == "4" and role == "admin":
                    hapus_data_admin()

                elif pilih == "5" and role == "admin":
                    tampilkan_daftar_user()

                elif pilih == "6" and role == "admin":
                    tampilkan_total_penggunaan()

                elif pilih == "7" and role == "admin":
                    proses_pengajuan_admin()

                elif pilih == "2" and role == "user":
                    tampilkan_total_user()

                elif pilih == "3" and role == "user":
                    ajukan_perangkat(username)

                elif pilih == "0":
                    print("")
                    print("Yah, Logout dulu ya")
                    input("Tekan Enter untuk melanjutkan...")
                    break

                else:
                    print("Yhaaaa Pilihannya gak ada, silahkan coba lagi")
                    input("Tekan Enter untuk melanjutkan...")
```

```
            except ValueError as e:
                print(f"[!] Error: {e}")
                input("Tekan Enter untuk melanjutkan...")

            except Exception as e:
                print(f"[!] Terjadi kesalahan: {e}")
                input("Tekan Enter untuk melanjutkan...")


if __name__ == "__main__":
    main()
```

4. **Hasil Output**

```
================================================
        WELCOME TO WATTWISE
   Monitor Ur Electricity Usage
================================================
Username : Admin
Password : Admin123

Login sebagai ADMIN. Halo, admin!
Tekan Enter untuk melanjutkan...
```

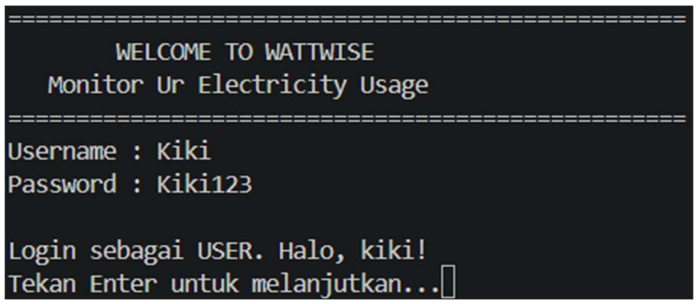
Gambar 4.1 Hasil Output Login sebagai Admin

```
================================================
   MENU ADMIN - admin
================================================
1. View Data Biling    4. Hapus Data Biling
2. Tambah Data Biling  5. Lihat Informasi User
3. Ubah Data Biling    6. Total Tagihan
7. Lihat Pengajuan User
0. Logout
================================================
Pilih:
```

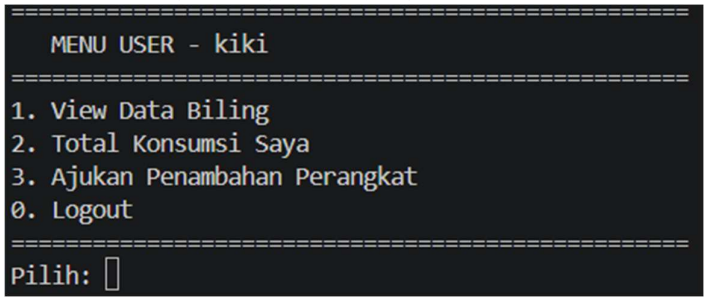
Gambar 4.2 Menu Output sebagai admin

```
=========================================================================
            DAFTAR PENGGUNAAN LISTRIK
=========================================================================
ID  Perangkat        Daya   Jam   Tanggal        kWh
-------------------------------------------------------------------------
1   Lampu   10W   5j   12-10-2026   0.05
2   Kipas Angin   50W   8j   12-10-2026   0.4
3   Kulkas   100W   24j   12-10-2026   2.4
-------------------------------------------------------------------------
TOTAL : 2.85 kWh  |  Rp 4117.4
```

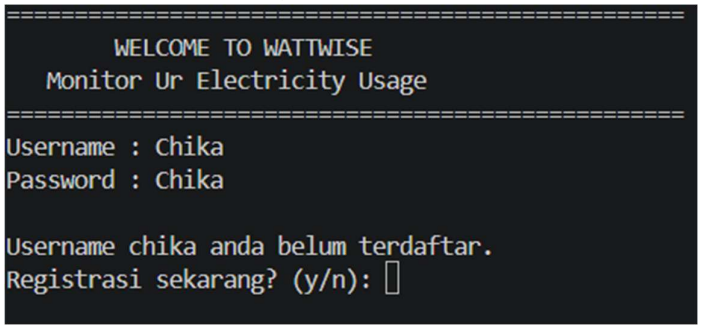
Gambar 4.3 Pilihan 1 Admin



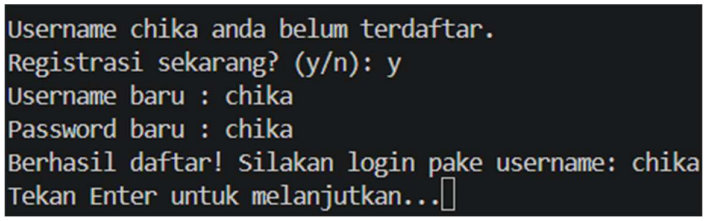
Gambar 4.4 Login sebagai User yang telah terdaftar



Gambar 4.5 Pilihan sebagai User



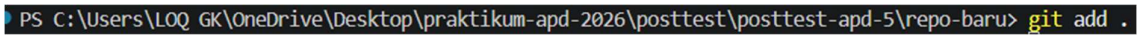
Gambar 4.6 Login sebagai user yang belum terdaftar



Gambar 4.7 Registrasi

5. Langkah-langkah Git

1.1.5. 1 Git Add



Perintah git add digunakan untuk menambahkan perubahan pada file ke dalam *staging area* sebelum disimpan ke dalam riwayat Git. Pada tahap ini, file yang telah dibuat atau mengalami perubahan dipersiapkan agar dapat diproses pada tahap *commit*. Perintah yang digunakan adalah git add . untuk menambahkan seluruh perubahan pada file yang terdeteksi di dalam direktori proyek.

git add .

Setelah perintah dijalankan, perubahan pada file akan masuk ke dalam *staging area*. Dengan demikian, file tersebut siap untuk disimpan ke dalam riwayat perubahan menggunakan perintah git commit.

1.2.5.2 Git Commit

```
PS C:\Users\LOQ GK\OneDrive\Desktop\praktikum-apd-2026\posttest\posttest-apd-5\repo-baru> git commit -m "posttest5.py"
```

Perintah git commit digunakan untuk menyimpan perubahan yang telah dimasukkan ke dalam staging area ke dalam riwayat repositori Git. Setiap commit memiliki pesan yang berfungsi untuk menjelaskan perubahan yang dilakukan. Pesan tersebut memudahkan pengguna dalam mengidentifikasi dan menelusuri riwayat pengembangan proyek.

Perintah yang digunakan adalah sebagai berikut.

git commit -m "Menambahkan fitur program"

Pada perintah tersebut, opsi -m digunakan untuk menambahkan pesan commit secara langsung. Setelah proses berhasil, perubahan akan tersimpan dalam riwayat lokal repositori Git, tetapi belum dikirim ke repositori jarak jauh.

1.3.5.3 Git Push

```
PS C:\Users\LOQ GK\OneDrive\Desktop\praktikum-apd-2026\posttest\posttest-apd-5\repo-baru> git push origin main
```

Perintah git push digunakan untuk mengirimkan *commit* yang tersimpan di repositori lokal ke repositori jarak jauh, seperti GitHub. Proses ini memungkinkan perubahan yang telah dibuat dapat tersimpan secara daring dan diakses melalui repositori yang terhubung.

Perintah yang digunakan adalah sebagai berikut.

git push origin main

Pada perintah tersebut, origin merupakan nama remote repositori, sedangkan main merupakan nama branch tujuan. Setelah proses *push* berhasil, perubahan yang telah di-*commit* akan dikirim ke repositori jarak jauh sehingga isi repositori tersebut diperbarui sesuai dengan *commit* yang dikirimkan.